

数据结构

您需要写一种数据结构来维护一些数，其中需要提供以下操作：

1. 插入 x 数
2. 删除 x 数(若有多个相同的数，因只删除一个)
3. 查询 x 数的排名(排名定义为比当前数小的数的个数+1。若有多个相同的数，因输出最小的排名)
4. 查询排名为 x 的数
5. 求 x 的前驱(前驱定义为小于 x ，且最大的数)
6. 求 x 的后继(后继定义为大于 x ，且最小的数)

有序表

根据上述操作，我们不难想到暴力的做法，因为我们需要知道一个数 x 在数列中的位置，已知他的前驱和后继，那么我们维护一个有序表就可以了。即数列是有序的，每次把数插入到对应的位置，我们来思考一下时间复杂度：

- (1) 插入数 x ：因为是有序表，所以我们可以考虑二分查找，找到位置的时间复杂度为 $O(\log n)$ ，但是每次插入后面的数又要向后挪一位，所以最坏时间复杂度为 $O(n)$
- (2) 删除数 x ：和插入相同。
- (3) 数 x 的排名：二分查找， $O(\log n)$
- (4) 排名为 x 的数：下标查找 $O(1)$
- (5) x 的前驱，二分查找， $O(\log n)$
- (6) x 的后继，同前驱。

有序表

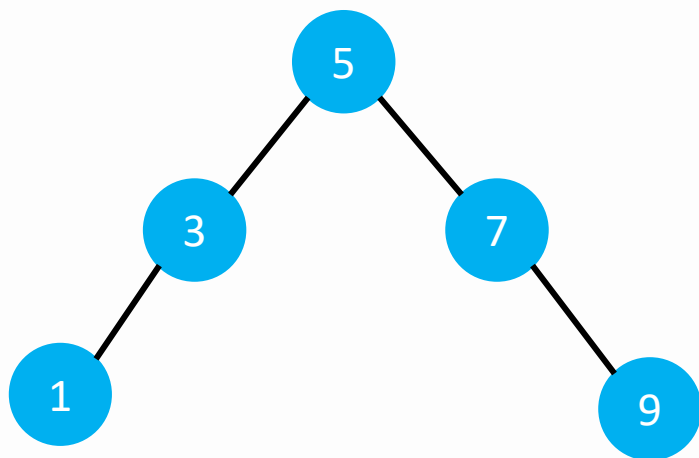
对于竞赛来说， $\log n$ 的算法就已经很优秀了，但是在插入和删除上的低效率就决定了被淘汰的命运。我们再思考一下如何在插入删除上优化，对于顺序表，插入和删除都是 $O(n)$ 的，并且又要可以使用二分的思想，所以我们只能考虑树形结构。

比如堆，堆的插入删除虽然是 $\log n$ 的，但是堆没有办法直接取中间任意元素，只能访问修改队首，也不符合我们的操作。

我们可以考虑来维护一颗有序的树，即该数的左子树一定比根结点小，右子树一定比根结点数大。这样当我们中序遍历这棵树的时候就得到了一个有序表。

有序表

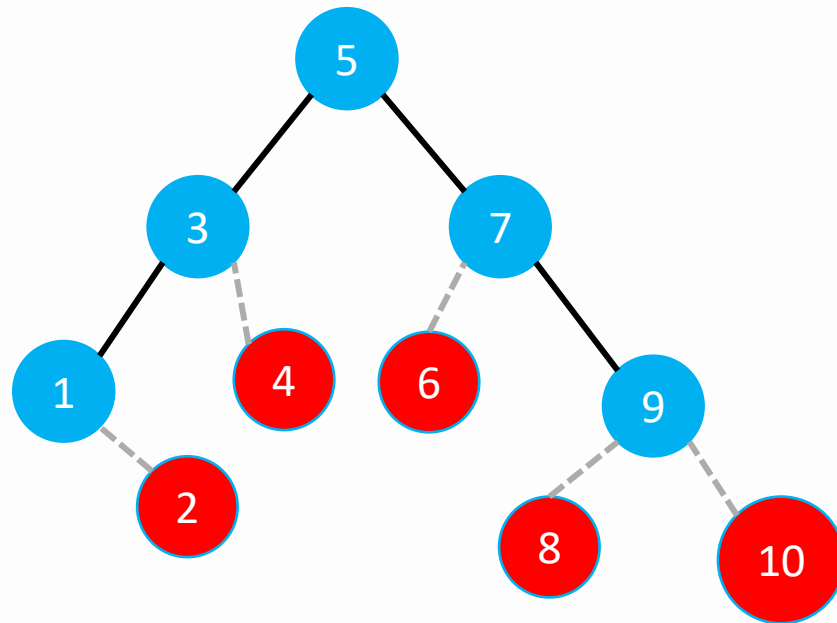
比如1 3 5 7 9，最开始的有序树的形态可能是：



中序遍历：1 3 5 7 9

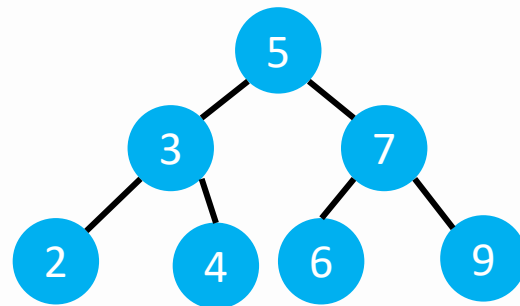
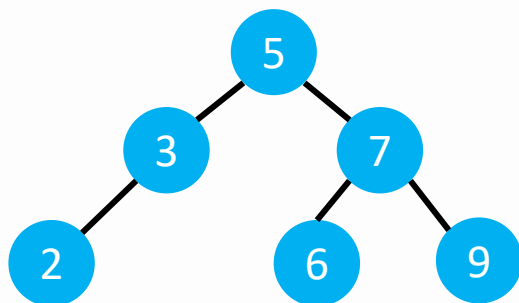
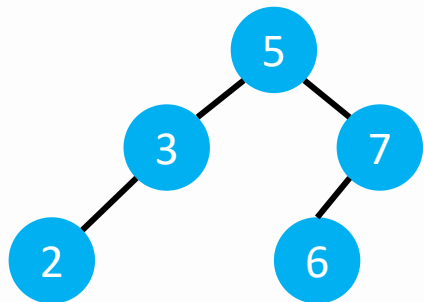
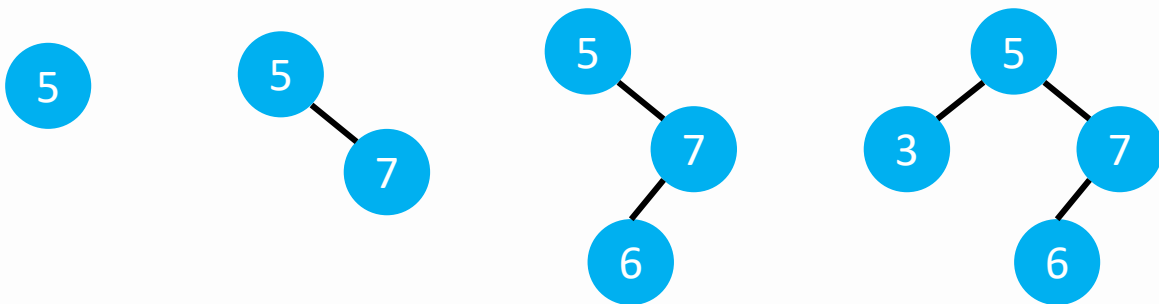
插入数2
插入数4
插入数6
插入数8
插入数10

有序表



有序树

比如 5 7 6 3 2 9 4，建树的过程为



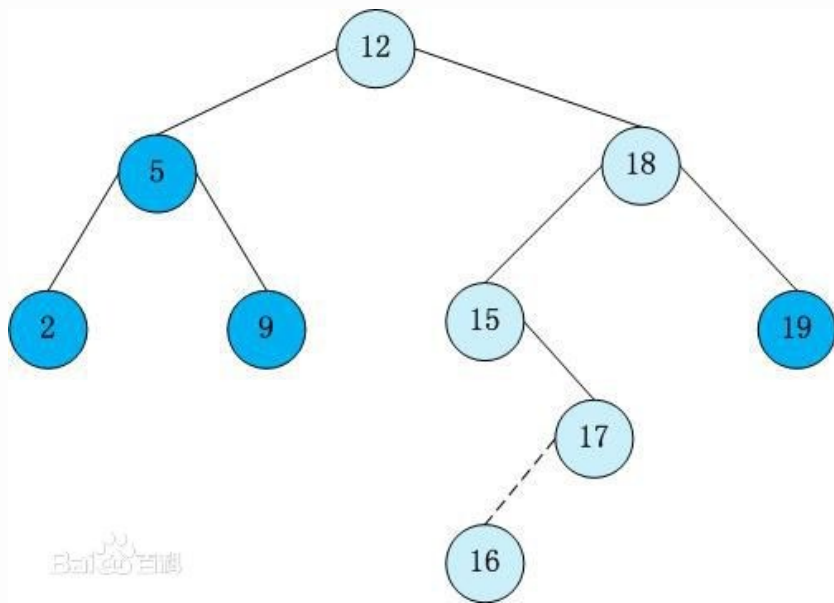
数据结构

您需要写一种数据结构来维护一些数，其中需要提供以下操作：

1. 插入 x 数
2. 删除 x 数(若有多个相同的数，因只删除一个)
3. 查询 x 数的排名(排名定义为比当前数小的数的个数+1。若有多个相同的数，因输出最小的排名)
4. 查询排名为 x 的数
5. 求 x 的前驱(前驱定义为小于 x ，且最大的数)
6. 求 x 的后继(后继定义为大于 x ，且最小的数)

二叉搜索树

二叉搜索树：又名二叉查找树，二叉排序树。具有以下性质：若他的左子树不为空，那么他的左子树上所有节点的值均小于根结点的值。若他的右子树不为空，那么他的右子树上所有节点的值均大于根结点。同样的，他的左右子树均满足二叉搜索树。



堆：根结点均大于(小于)左右子树的完全二叉树。

应用

因为二叉搜索树的左子树均小于他的根，右子树均大于他的根，所以具有很强的实际应用。

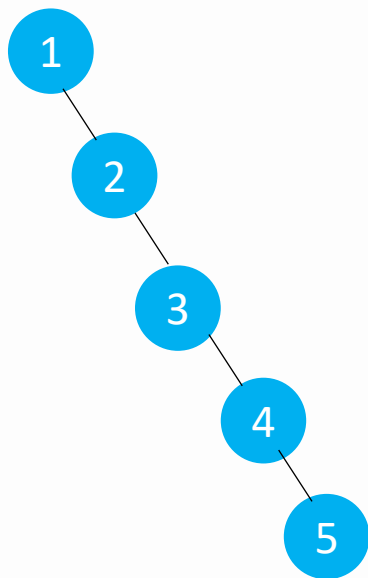
- 1: 排序
- 2: 查找第k大的值。
- 3: 判断一个数是第几大的数。
- 4: 修改并查询

由于它是有顺序的树形结构，我们可以通过二分的办法很快对该树进行搜索。理论上的时间复杂度为 $O(n \log n)$ 。但是！！！！

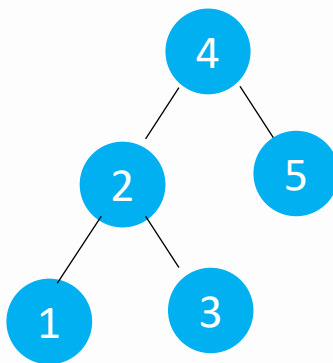
二叉搜索树

我们在实际操作中会发现一个问题——二叉搜索树的建立不是唯一的。他的形态取决于我们建树的方法。如果按照一般情况下，边输入边维护二叉搜索树的办法建树，会存在一定的问题。

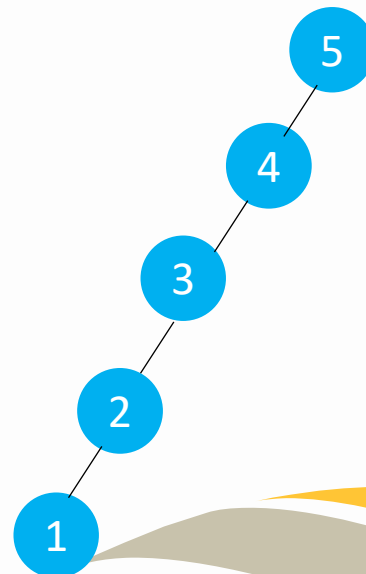
1 2 3 4 5



4 5 2 1 3



5 4 3 2 1



不足之处

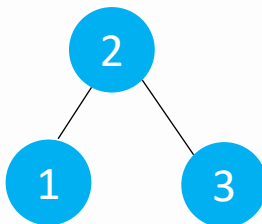
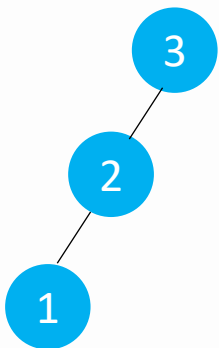
从上图可以看出，我们建立的二叉搜索树的形态取决于数据输入的顺序，如果每次输入的数相对于根结点来说比较中间或者使用随机数，生成的二叉搜索树是比较接近完全二叉树的，时间复杂度就比较低，查找的时间复杂度近似为 $O(n \log n)$ 。但是如果输入的数据大部分都是有序的，那么生成的二叉搜索树形态就比较接近一条链，甚至就是一条链。搜索效率肯定就比较低，还不如直接用线性表。所以直接使用二叉搜索树在竞赛中很难得到高分。

通过观察我们会发现，如果一棵二叉搜索树的左右子树的数量越相等，查找效率就越高。即如果一棵二叉搜索树越平衡，查找效率就越高。所以我们要保证二叉搜索树的平衡性。这样的二叉搜索树叫平衡二叉搜索树。简称平衡树。

平衡树

维护二叉搜索树平衡的方法各种各样，不同的平衡树维护的方法也不相同，比如SBT，AVL，红黑树，替罪羊树，Treap，Splay，FHQ Treap等。总体上来说分为旋转维护和不旋转维护两种。

简单来说，就是通过修改树上的父子关系来让一个二叉搜索树尽可能的平衡。



非旋Treap

我们前面已经说过，二叉搜索树的效率会根据给定的数据发生改变，特别是给定的数据大部分有序的情况下。那我们不就很容易想到一种提高效率的办法：随机化

也就是说，如果事先给出了数据，那么我们完全可以先把数据随机化一遍再建树。但是很多情况下，都是边给操作边给询问，就没有办法这样做了。

那我们就考虑另一种办法——给每个结点添加一个随机数(优先级)，即此时每个结点有两个关键字，一个是权值，满足二叉搜索树的性质，一个是随机优先级，并且通过一系列操作让他满足小根堆的性质，这在这些操作的过程中，由于优先级是随机的，所以会让这棵树比较平衡。

这就是Treap的原理，tree(二叉搜索树)+heap(堆)

让这颗二叉搜索树满足堆的性质有多种，有带旋转的，有不带旋转的，这里主要讲解非旋Treap(FHQ平衡树)

非旋 Treap

在学习FHQ之前，我们要先学习左偏树——可并堆，因为他的主要操作有两个，一个是合并(merge)，一个是分裂(split)，本质上就是堆的合并和分裂，具体实现和可并堆(左偏树)比较类似。

我们先来观察一下，两个孤立结点的合并。

key: 权值

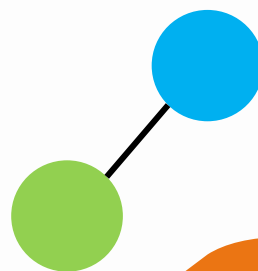
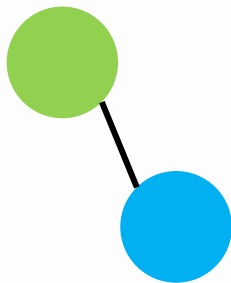
val: 优先级

key=1
val=1

key=2
val=2

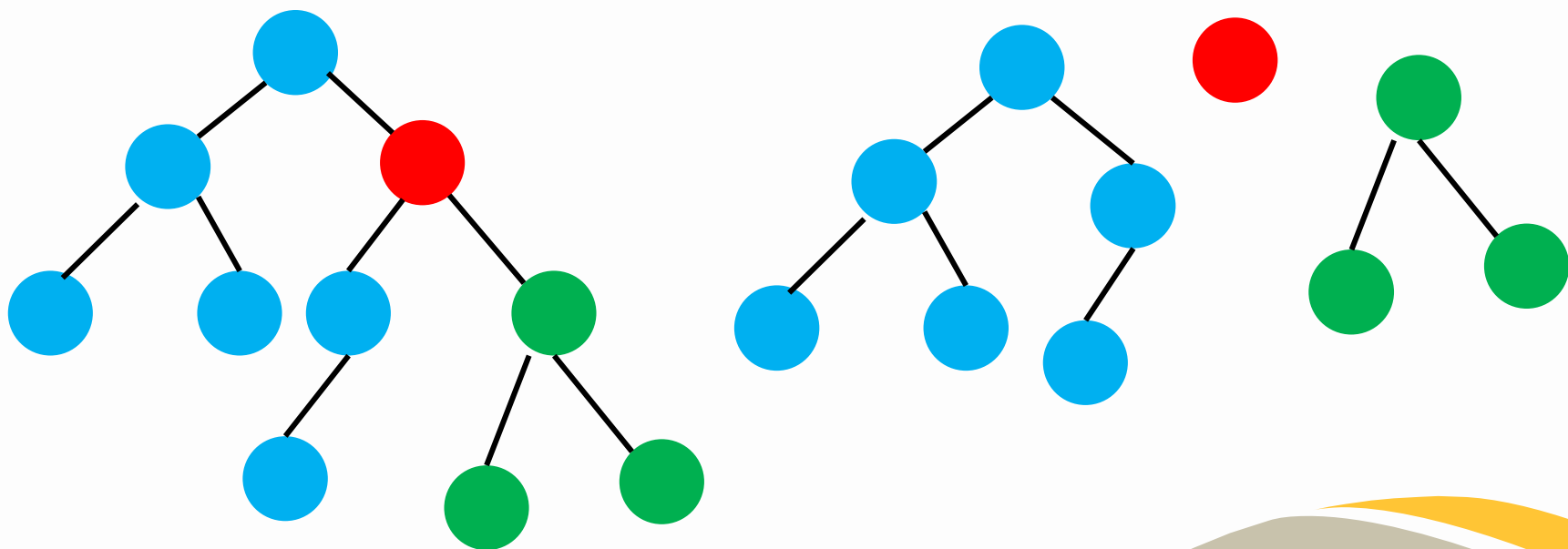
key=1
val=2

key=2
val=1



拆分

FHQ的拆分很简单，假设我们需把某个点 x 删除或者把排名为 x 的点删除，那么我们就把这一颗树拆分成三个部分，第一个部分是比较 x 小的一棵树，第二个部分是比较 x 大的一棵树，最后一个部分就是 x 这个结点。



拆分

假设我们要删除权值为 x 的点：

(1) 从根结点开始往下找，如果当前根结点 $\leq x$ ，那么就说明该根结点的左子树都是 $\leq x$ 的，并且该根结点就是拆分之后的左子树的根，该根结点的右子树还没有确定。如果当前根结点 $> x$ ，那么说明该根结点的右子树都是 $> x$ 的，并且该根结点就是拆分之后的右子树的根，该根结点的左子树还未确定。

(2) 如果满足根结点 $\leq x$ ，那么我们就继续去右子树里面找，也就是拆分右子树，并且把当前右子树中比 x 小的树链接到根结点的右子树中去

(3) 当遍历结束的时候，返回。

(4) 返回的时候更新当前根结点的左右子树的数量

拆分

```
void split(int u,int x,int &l,int &r)//l,r分别表示当前子树根
{
    if(!u)//没有子树可以拆分了, 说明该结点的左右子树都为空
    {
        l=r=0;
        return ;
    }
    if(tree[u].key<=x)//如果x在当前根结点的右子树
    {
        l=u;//左子树的根确定了
        splid(tree[u].r,x,tree[u].r,r);//继续拆分右子树
    }
    else//如果x在当前根结点的左子树
    {
        r=u;//右子树的根确定了
        splid(tree[u].l,x,l,tree[u].l);//继续拆分左子树
    }
    update(u);//更新根结点的size
}
```

删除

注意，一次拆分是把一棵树拆成两颗子树，但是删除的时候我们最终的目的是拆成三棵树，没关系，拆两次就可以了，因为第一次是 $\text{tree}[u].\text{kay} \leq x$ ，那么说明 x 最终一定在左子树里面，所以我们再把左子树拆一遍就可以了。

```
split(root,x,l,p);  
split(l,x-1,l,r);//按照x-1拆, >x-1的只有x  
r=merge(tree[r].l,tree[r].r);//可能权值为x的不止一个  
root=merge(merge(l,r),p);//再合并, 注意可能还有三棵树
```

合并

我们想一下，合并的时候，有两种合并方式，第一种是按照权值合并，第二种是按照优先级合并，（都要保证另一个也成立）都是可以我们这里按照优先级合并，也就是可并堆的合并方式。

如果按照优先级合并，说明优先级小的为根，大的为子树。此时我们又知道，左子树 $<$ 右子树的，所以：

(1) 如果左子树根结点优先级 $\leq$ 右子树根结点优先级，那么左子树根结点合并的根，右子树的根结点移动到左子树的右子树中去，这样能保证二叉搜索树的性质

(2) 如果左子树根结点优先级 $>$ 右子树根结点优先级，那么右子树就是合并后的跟，左子树的根结点移动到右子树的右子树中去，这样仍然能保证二叉树性质。

合并

```
int merge(int l,int r)
{
    if(!l||!r)//如果其中一个为空, 直接把另一棵树接上去
        return l+r;
    if(tree[l].val<tree[r].val)//说明左子树为根
    {
        tree[l].r=merge(tree[l].r,r);//左子树的右孩子可能会变
        update(l);//更新size
        return l;//返回合并后的根结点
    }
    else//说明右子树为根.
    {
        tree[r].l=merge(l,tree[r].l);//右子树的左孩子可能会变
        update(r);
        return r;//返回合并后的根结点
    }
}
```

插入

因为前面也是把 x 放入左子树中，所以我们这里也一样，左子树中可能包括数 x ，所以我们先按照 x 把一棵树分成左右子树，左子树 $\leq x$ ，右子树 $> x$ ，然后再插入一个结点，该结点的值为 x ，最后再合并这三棵子树。

```
void add(int x)
{
    cnt++;
    tree[cnt].size=1;
    tree[cnt].val=x;
    tree[cnt].key=rand();//精髓所在
    tree[cnt].l=tree[cnt].r=0;//此时是孤立结点
}
```

```
split(root,x,l,r);
add(x);
root=merge(merge(l,cnt),r);
```

查询x的排名

要知道x的排名，只需要知道 $\leq x$ 的数有多少个就可以了，直接分裂就可以了，因为split里面 $\leq x$ 是一起的，所以我们分裂的值选择 $x-1$ 。然后x的排名就是左子树的大小+1。但是一定要注意，分裂之后一定不要忘记合并。

```
split(root,x-1,l,r);  
printf("%d\n",tree[l].size+1);  
root=merge(l,r);
```

查询排名x的数

对于一棵二叉搜索树，如果要查找第x大的数，直接根据根结点的信息查询就可以了，如果 $x \leq \text{根结点左子树.size}$ ，说明x肯定在左子树中，递归查找左子树，如果 $x = \text{根结点左子树.size} + 1$ ，x就在根结点，否在x就在右子树，但是此时在右子树中的排名就是 $x - \text{左子树.size} - 1$ 。

```
int kth (int u,int k)
{
    if(k<=tree[tree[u].l].size)//说明在左子树中
        return kth(tree[u].l,k);//递归查找左子树
    else
    {
        if(k==tree[tree[u].l].size+1)//说明就在根结点
            return u;//返回根结点
        else
        {
            k-=tree[tree[u].l].size+1;//右子树只是当前子树的排名，要减去前面的
            return kth(tree[u].r,k);//递归查找右子树
        }
    }
}
```


求x的前驱

先把一棵树按照 $x-1$ 分成两棵子树，那么x的前驱的排名就是左子树的大小，所以再用kth函数查询即可

```
split(root,x-1,l,r);  
printf("%d\n",tree[kth(1,tree[l].size)].val);  
root=merge(l,r);
```

求x的后继

先把一棵树按照 $x-1$ 分成两棵子树，那么x的后继的排名就是右子树中最小的那一个。

```
split(root,x,l,r);  
printf("%d\n",tree[kth(r,1)].val);  
root=merge(l,r);
```

文艺平衡树

您需要写一种数据结构来维护一个有序数列，其中需要提供以下操作：翻转一个区间，例如原有序数列是5 4 3 2 1，翻转区间是[2,4]的话，结果是5 2 3 4 1

输入：

5 3

1 3

1 3

1 4

输出：

4 3 2 1 5

$n, m \leq 100000$

文艺平衡树

这道题简单来说就是维护一颗支持区间修改的平衡树，我们来思考一下FHQ平衡树如何支持区间操作。

比如我要翻转区间 $[l, r]$ ，那么很明显我需要先把区间 $[l, r]$ 用一颗单独的树提取出来，所以要用到split操作：

先分裂成 $[1, r]$ 和 $[r, n]$

`Split(root, r, l, r);`

再把子树 l 分裂成 $[1, r-1]$ 和 $[r, l]$

`Split(l, r-1, l, p)`

那么此时以 p 为根结点的子树就表示区间 $[l, r]$ ；

区间分割

但是值得注意的是，以前我们是按照权值大小构造平衡树，维护的也是权值的大小。但是这里我们是翻转的区间，所以我们应该维护的是区间的编号，所以Split函数应该按照size来划分，而不是val。

```
if (tree[tree[u].l].size+1<=x) //注意是按照区间分割
{
    l=u;
    split(tree[u].r, x-tree[tree[u].l].size-1, tree[u].r, r);
    //如果在右边，要减去前面的部分
}
else
{
    r=u;
    split(tree[u].l, x, l, tree[u].l);
}
update(u);
```

标记下传

但是因为是区间修改，所以我们不能全全部修改，而是应该像线段树一样，考虑区间整体修改，生成lazy标记。如果当前根结点有lazy标记，并且需要遍历下面的子树，就把当前根结点的lazy标记下传，并且清空当前lazy标记。

我们再考虑一下区间翻转，实际上对于一颗二叉搜索树来说，区间翻转实际上就是递归交换左右子树。当交换到根结点的时候，那么就实现了区间的翻转。

标记下传

```
void pushdown(int u)
{
    int tmp=tree[u].l; //交换左右子树
    tree[u].l=tree[u].r;
    tree[u].r=tmp;
    tree[tree[u].l].lazy^=1; //更新标记
    tree[tree[u].r].lazy^=1;
    tree[u].lazy=0; //清空标记
}
```

平衡树输出

平衡树的输出实际上就是平衡树的中序遍历，但是一定要注意lazy标记下传。

```
void prit(int u) //中序输出
{
    if(tree[u].lazy)
        pushdown(u);
    if(tree[u].l)
        prit(tree[u].l);
    printf("%d ", u);
    if(tree[u].r)
        prit(tree[u].r);
}
```